

ACADEMIC-OS

DEFINITION DOCUMENT

Version 7 — Comprehensive Edition

University of Pretoria | sip.zuma@gmail.com | enathicoki@gmail.com

Table of Contents

- 1. Introduction**
 - 1.1. Purpose
 - 1.2. Background
 - 1.3. Scope
 - 1.4. Definitions, Acronyms & Abbreviations
 - 1.5. References
 - 2. General Project Description**
 - 2.1. System Overview & Context
 - 2.2. Architectural Evolution — V1 vs V7
 - 2.3. State Management & Analytics Engine
 - 3. Requirements & Constraints**
 - 3.1. Functional Requirements
 - 3.2. Non-Functional Requirements
 - 4. Use Cases**
 - 4.1. Use Case Glossary
 - 4.2. Deep-Dive Use Cases
 - 4.3. Fully Attributed Entity Relationship Model
 - 5. Engineering Standards & Guardrails**
 - 6. Conclusion & Outstanding Issues**
-

1. Introduction

1.1. Purpose

This Requirements Definition Document serves as the comprehensive reference for the development and implementation of Academic-OS. Fully updated to Version 7, it supersedes all initial references to the v1 Progressive Web App (PWA). It defines what the system must do, how it must perform, and the constraints that govern its development across all native deployment targets — Desktop (Electron) and Mobile (Capacitor).

This document is intended for:

- The development team building and maintaining Academic-OS.
- Stakeholders who need to understand system capabilities and design decisions.
- Future maintainers who will extend or refactor the system.

NOTE: This is a living document, refined continuously during development. Version 7 drastically changes the technical landscape — moving away from a browser-only PWA into an integrated, offline-first Electron and Capacitor architecture with Clean Architecture patterns throughout.

1.2. Background

Where did it all start?

Academic-OS emerged from a real need: managing academic performance should not require complex manual spreadsheets. The system was inspired by a previous Excel-based tracker that, while functional, lacked automation, real-time insights, and active mobile synchronisation capabilities.

What you need to know about Academic-OS — Version 7 Updates

The academic environment at the University of Pretoria (and universities broadly) presents students with compounding challenges. V7 was specifically evolved to address:

- Multiple assignments across modules with varying weighting algorithms — accommodating strict, balanced, or continuous assessment configurations.
- Granular performance trend tracking powered by the newly introduced **TileFormulaEvaluator** and incremental Analytics Engine.
- Differentiated task strategy between rapid Sprints and long-term Marathons to provide targeted focus loops.
- Centralised cross-platform usage via native OS capabilities without relying on a cloud database for primary operation.

What makes Academic-OS indispensable is its focus on actionable insight. Students need to know:

- Where things are going wrong academically — incomplete module tasks, failing grades, overdue items.

- What their capacity is versus required effort — the Effort vs. Capacity Ratio.
- Which tasks or assessments demand immediate attention and how much time that will realistically cost.

1.3. Scope

This section defines what Academic-OS will and will not do in its v7 iteration.

In Scope:

- Management of a single student's complete academic data across their entire degree programme.
- Cross-platform native binaries: **Electron 36 (Desktop)** and **Capacitor 7 (Android/Mobile)**, built from a unified React 19.x frontend.
- Performance tracking and analytics for:
 - Modules — incorporating integrated Blueprint + Offering composite models.
 - Assessments — tests, assignments, projects, and exams (now decoupled as first-class entities).
 - Tasks — time management, effort tracking, calculated time spent, and focus loops.
 - Goals — term-level GPA targets, weekly/term time-budget goals, and semester-level milestones.
- Grade calculations showing current standings, required exam marks to hit targets (accounting for subminimum rules), and pre-calculated gap-to-target metrics.
- Comprehensive Notification and Alert system integrated natively into the host OS environment.
- Data import via CSV files, including mapped previews and template downloads.
- Explicit Database Backup and Sync mechanism via user-driven **Google Drive Sessions**.
- Fully offline operation — SQLite via WASM for Desktop, Capacitor-Community SQLite for Android.

Out of Scope (Current Release):

- Real-time multi-user collaboration or social sharing features.
- Direct automatic integration or scraping from university LMS systems (e.g., Blackboard, Canvas).
- Prerequisite tracking strict enforcement — deferred to simplify the UI while the data property remains intact for future implementation.

1.4. Definitions, Acronyms & Abbreviations

Key Definitions:

- **Assessment:** Graded academic work including tests, assignments, practicals, projects, and continuous evaluations.

- **Exam (NEW):** Decoupled first-class entity. Evaluated as a separate weight matrix within a Module, enabling distinct entrance requirement and subminimum calculations.
- **Task (NEW):** Granular study activity or preparation work categorised as Sprint (short-burst) or Marathon (sustained multi-week effort), with logged time costs.
- **Module:** A course within a degree programme — composites a timeless Blueprint with a term-specific Offering record.
- **Goal / Horizon:** A target GPA, study-hour budget, or performance milestone set at day, week, module, or semester scope.
- **Mark Range / Effort Spikes:** Algorithmic predictions that identify weeks where assigned task hours dangerously exceed a student's available capacity.
- **Dashboard / Horizon View:** Filterable timelines presenting immediate priorities across scopes: Day, Week, Month, Term, and All.
- **Value Object:** Immutable domain wrappers (e.g., GradePercentage, ModuleCode) that enforce strict validation and prevent raw-primitive bugs in domain logic.
- **TileFormulaEvaluator:** The incremental analytics engine that computes and caches all calculated_ fields without full-tree recalculations.

Acronyms and Abbreviations:

- **Academic-OS:** Academic Operating System
- **CSV:** Comma-Separated Values
- **DB:** Database
- **UI / UX:** User Interface / User Experience
- **GPA:** Grade Point Average
- **DP:** Duly Performed — the semester/class mark prerequisite to write a final exam
- **WASM:** WebAssembly — used to run SQLite natively in Electron and browser contexts
- **HLC:** Hybrid Logical Clock — used for conflict detection during multi-device cloud sync
- **NQF:** National Qualifications Framework

1.5. References

- Sipho Zuma — sip.zuma@gmail.com (Domain Expert)
- Enathi Coki — enathicoki@gmail.com (Domain Expert)
- Internal API & State Schema — src/core/domain/models/, src/types/
- University academic regulations and NQF levelling guidelines.
- Clean Architecture by Robert C. Martin — foundational architectural reference.

2. General Project Description

2.1. System Overview & Context

Academic-OS operates as a standalone personal academic management system. Built on Clean Architecture patterns, it serves as a student's private analytical engine — processing

academic data to produce actionable insights on performance, workload capacity, and goal achievement, entirely without relying on cloud computation for core functionality.

Key Architectural Layers:

- **Presentation Layer:** React 19.1 with Vite 6.2. Uses Framer Motion and Vault Sheets for premium interactions. Custom stability hooks (useStableCallback, loop detection) ensure 60fps rendering even on low-end Android hardware.
- **State Management (Zustand + Immer):** The single source of truth for UI reactivity. Normalised stores (ModulesStore, AssessmentStore, SettingsStore) with explicitly cached selectors prevent unnecessary re-renders.
- **Application Layer:** Strict CommandHandlers orchestrate all business operations, ensuring domain rules never bleed into UI components.
- **Domain Layer:** The system's analytical core. Rich Domain Models (Module, Assessment, Term, Degree, Task) and rigid Value Objects isolate all grade calculations, projection logic, and effort analysis here.
- **Data Access Layer:** Repository pattern over a Unit of Work prevents incomplete writes and abstracts all SQLite query logic from higher layers.
- **Storage Layer:**
 - **Desktop / Web** — sql.js (SQLite compiled to WebAssembly).
 - **Android / Mobile** — @capacitor-community/sqlite bridging native device SQLite.

Core System Philosophy — Three Questions Academic-OS Always Answers:

- **What is my immediate workload?** Tasks, priorities, Sprint vs Marathon tracking.
- **How am I performing?** Module grades, Term GPA, trend calculations.
- **Am I meeting my goals?** Progress vs required average to pass or achieve targets.

2.2. Architectural Evolution — V1 vs V7

Academic-OS has evolved from a simple spreadsheet-replacement PWA into a highly structured, natively-executing local application. The table below summarises the key paradigm shifts.

Property	Detail
Deployment Target	V1: Progressive Web App accessed via a browser. V7: Compiled Electron 36 binary (Desktop) and Capacitor 7 APK (Android) — true native execution.
Database	V1: Volatile browser IndexedDB / localStorage. V7: Real relational SQLite — WASM (Desktop) and native Capacitor plugin (Mobile).
State Management	V1: Ad-hoc React state and prop-drilling. V7: Centralised Zustand stores with Immer-drafted immutable updates.
Analytics	V1: Grades computed inside React components on every render. V7: Decoupled Analytics Engine (TileFormulaEvaluator) persists calculated_ fields to the DB; UI only reads pre-computed values.

Exam Handling	V1: Exam treated as a standard Assessment. V7: Exam is a decoupled entity with independent weight matrix, entrance requirements, and subminimum rules.
Task Model	V1: No discrete task concept. V7: Tasks are first-class entities with Sprint/Marathon strategy, time tracking, and calculated effort-vs-capacity metrics.
Settings	V1: UI-only toggles stored in localStorage. V7: SystemSettings is a persisted Domain Entity with version tracking, fine-grained notifications, and filter modes.
Cloud Sync	V1: None. V7: Explicit Google Drive session sync using HLC timestamps, with graceful conflict detection.

2.2.1. The Shift to Offline-First Native Execution

V7 completely abandons the browser as the primary runtime:

- **Desktop (Electron 36):** The application compiles to a native binary with access to Node.js APIs (fs, path, crypto) for secure file writing and cryptographic operations during Drive sync. Users interact with a full desktop window, not a browser tab.
- **Mobile (Capacitor 7):** The app runs as a true native Android APK, accessing persistent device storage instead of volatile browser IndexedDB. Hardware back-button routing and native Action Sheets (Vaul) deliver an authentic mobile experience.
- **Database Migration:** From ephemeral browser storage to a real relational SQLite database — enabling complex JOIN queries, forward-migrating schemas, and reliable multi-year data retention.

2.3. State Management & Analytics Engine

To eliminate UI lag and grade miscalculations inherent in V1, V7 introduces a rigid, decoupled analytical tier.

2.3.1. Zustand + Immer State Architecture

- **Centralised Stores:** All application state lives in dedicated Zustand stores — ModulesStore, AssessmentStore, SettingsStore, etc.
- **Immutable Updates:** Immer allows drafted mutations that produce new immutable state objects, preventing accidental UI mutation bugs across deeply nested academic data structures.
- **Cached Selectors:** Explicit memoised selectors prevent components from re-rendering when unrelated slices of state change — critical for large assessment histories.

2.3.2. The Analytics Engine (TileFormulaEvaluator)

The Analytics Engine represents one of V7's most significant engineering achievements:

- **Incremental Calculations:** Rather than recalculating the entire grade tree on every change, the engine identifies which `calculated_` fields are dirty and recomputes only those.
 - **Persisted Results:** All analytical values (GPA, projected grades, exam entrance gaps, effort ratios) are written back to the SQLite database as `calculated_` columns — not held in memory.
 - **UI Read-Only:** The Presentation Layer never performs grade arithmetic. It exclusively reads pre-computed values, guaranteeing 60fps performance even with years of accumulated academic history.
 - **TermNavigatorData:** A complex pre-aggregated object supplied by the backend that powers the Horizon Dashboard — effort timelines, GPA trends, and workload spikes — without any client-side computation.
-

3. Requirements & Constraints

3.1. Functional Requirements

Data Management

1. System allows full CRUD operations for degrees, terms, modules, assessments, tasks, and system settings.
2. All data modification flows through the Command/Query pattern — no direct UI-to-database writes.
3. Full support for importing module and assessment data via CSV, including template generation and column mapping.
4. System supports cross-platform data synchronisation via user-authenticated Google Drive sessions.
5. The system maintains forward-migrating schemas (e.g., `005_add_hlc_columns.ts`) to prevent data loss on version upgrades.

Data Analysis & Calculations

1. The system uses an incremental calculation engine so analytical metrics (GAP to Target, Required Average, Weekly Effort) are persisted as `calculated_` fields, not computed in the UI.
2. The UI never recalculates complex graphs — it relies entirely on the pre-computed `TermNavigatorData` object from the backend query.
3. The system handles subminimum and exam entrance requirement logic distinctly from continuous assessment coursework mathematics.
4. Accurate grade composition handling for both continuous assessment modules and standard exam-weighted modules.

User Interface & Interaction

1. The Dashboard serves as the cross-module command centre, surfacing the most urgent tasks and grade statuses.

2. System leverages native-feeling mobile components — Action Sheets via Vault — and correct Android hardware back-button routing.
3. Visual smoothness is guaranteed: Create Category and Add Assessment interactions respond instantly before committing to SQLite.
4. The Module Details view gracefully scales on tablets, preventing header-padding overlap bugs.
5. Comprehensive Settings component allowing Theme, Language, Notification intensity, and Sync configuration.

3.2. Non-Functional Requirements

Performance & Stability

1. Dashboard must render within 3 seconds, avoiding white or un-themed flash during initialisation.
2. Form fields must respond without input lag — handled via decoupled text input components that prevent whole-sheet re-renders.
3. UI guarded by strict React stability hooks (useStableCallback) to prevent render loops in complex assessment tables.
4. System handles multi-year assessment histories without degrading frame rates.

Usability

1. System must support touch (Android) and mouse/keyboard (Electron Desktop) natively, adapting to the active platform.
2. Automatic dark/light theme switching based on OS preferences, applied before first render to prevent visual tearing.
3. Bottom sheets must resize dynamically when the on-screen keyboard appears, without animation stuttering.
4. Onboarding wizard makes the system learnable within 30 minutes for a first-time student user.

Reliability & Security

1. Completely offline-capable — no feature is gated behind an internet connection except explicit remote sync.
2. Schema structure backups export correctly to JSON/SQLite matching the exact v7 schema structures.
3. If a Google OAuth token expires mid-sync, the user is prompted gracefully without infinite spinners or data corruption.
4. All user data remains encrypted or strictly local. No central server telemetry is tracked.

4. Use Cases

4.0. Use Case Diagram — V7 Overview

The diagram below presents the complete Version 7 use case model, updated from the V1 baseline. It captures all actors, system boundaries, and relationships — including the newly introduced Task Management section (Section 3) and Cloud Synchronisation use case (1.6).

ACADEMIC-OS — Use Case Diagram (Version 7)

Updated from V1 · Actors: Student (PBA), Temporal (PBA) · System: Academic-OS

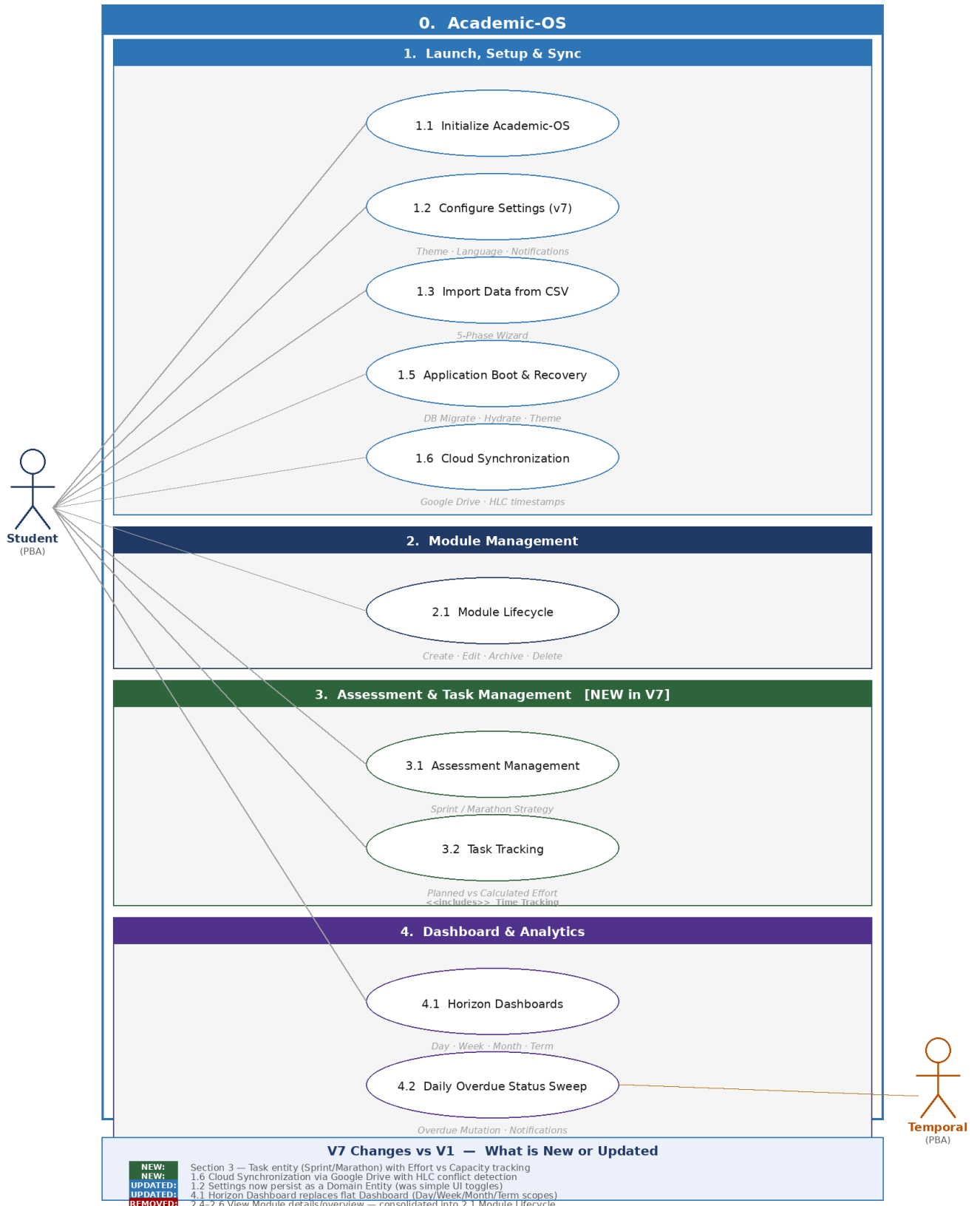


Figure 1: Academic-OS V7 Use Case Diagram (updated from V1 baseline)

NOTE: The editable .drawio source file (Use_Case_Diagram_v7.drawio) is provided separately and can be opened in draw.io Desktop or diagrams.net. Section 3 (Task Management) is entirely new in V7. V1 use cases 2.4-2.6 were consolidated into 2.1 Module Lifecycle.

4.1. Use Case Glossary

Use-Case Name	Description	Actor
1.1 Initialize Academic-OS	First-time setup defining degree structure and initial goals.	Student
1.2 Configure Settings (v7)	Modifying Application Language, Theme, Notification Intensity, Filter modes, and Backup reminders within a persistent SystemSettings singleton.	Student
1.3 Import Data from CSV	A 5-phase wizard (Download Template → File Selection → Column Mapping → Conflict Resolution → DB Commit) mapping tasks/assessments into the DB.	Student
1.5 Application Boot	A complex async state machine: DB load → Zustand hydration → DB migrations → boot-flash prevention.	System
1.6 Cloud Synchronization	Manual sync up/down from Google Drive to backup the SQLite instance, verifying token validity first.	Student
2.1 Module Lifecycle	Creating, Editing, Archiving, or Deleting a Module. Modifying continuous/exam grading structures visually.	Student
3.1 Assessment / Task Management	Adding Sprint or Marathon Tasks to specific parent Assessments. Tracking effort costs and updating states dynamically.	Student
4.1 Horizon Dashboards	Viewing 10,000-foot views across Time Horizons (Day, Week, Month) to judge total effort-ratio capacity.	Student
4.2 Daily Overdue Sweep	Time-based actor triggered internally that flips items to Overdue if deadlines pass.	Temporal

4.2. Deep-Dive Use Cases

The following use cases document the complex invisible machinery that makes V7 robust, reliable, and performant. Each is presented with its primary actor, goal, and detailed flow.

4.2.1. Use Case: The V7 Boot Sequence (App Initialisation)

Property	Detail
----------	--------

Primary Actor	System (automated on application launch)
Goal	Load the database, apply any pending migrations, hydrate the UI state, and apply the user's theme — all before the first pixel is painted — to prevent blank screens or data corruption.

Flow:

1. The AppShell component mounts and initiates the BootManager service.
2. The Database Service is initialised — WASM is loaded for Desktop or the Capacitor SQLite bridge is established for Android.
3. **Idempotent Migrations:** The system checks the current database schema version against the code version. It sequentially runs any missing migration files (e.g., 005_add_hlc_columns.ts) to upgrade the local database without losing user data.
4. **Hydration:** Raw SQLite rows are pulled, passed through Domain Model Factories (e.g., Module.fromData()), and injected into the Zustand global store.
5. **Theme Mount:** The saved user theme (light/dark/system) is applied before the UI renders to prevent a white-screen flash on first paint.

NOTE: Steps 3–5 execute in strict sequence. A failure at the migration step halts boot and surfaces a recovery dialog — preventing the user from interacting with a partially migrated database.

4.2.2. Use Case: Granular Workload Management (The Horizons)

Property	Detail
Primary Actor	Student
Goal	Understand immediate capacity versus required effort without being overwhelmed by the full degree workload.

Flow:

1. Student navigates to the Assessment / Horizon View.
2. The UI queries the TermNavigatorData pre-computed object from the Zustand store (no live calculation).
3. **Time Horizon Filtering:** The system presents tasks filtered by scope:
 - **Day:** What must happen today?
 - **Week:** What is the effortVsCapacityRatio for the next 7 days?
 - **Month / Term:** What does the aggregate grade trend look like across all modules?
4. **Effort Spike Detection:** The view highlights weeks where assigned task hours exceed the student's available capacity — surfacing overcommitment before it causes missed deadlines.
5. Student can promote or demote items between Sprint and Marathon strategies directly from this view, triggering a background recalculation.

4.2.3. Use Case: Cloud Synchronisation via Google Drive

Property	Detail
Primary Actor	Student (triggered) & System (executed)
Goal	Back up the local SQLite database to Google Drive and enable data transfer between Desktop and Mobile devices.

Flow:

1. Student triggers Sync Now from the Settings screen.
2. The system checks for a valid Google OAuth token — handling Electron deep-link callbacks or Capacitor OAuth intents natively.
3. **Conflict Detection:** The system compares the local database's Hybrid Logical Clocks (HLC) or simple timestamps against the cloud backup metadata.
4. The system pushes a securely exported JSON payload or raw .sqlite binary to a dedicated AppData folder in the user's Google Drive.
5. **Conflict Resolution:** If the user has edited data on their phone and PC while both were offline, the system applies a last-write-wins block replacement strategy, or alerts the user to a schema mismatch requiring manual resolution.
6. On successful sync, lastBackupDate is updated in SystemSettings and a confirmation notification is dispatched.

KNOWN LIMITATION: The current last-write-wins strategy is acknowledged as a simplification. Granular event-sourcing merge conflict resolution (e.g., vector clock CRDT logic) is a roadmap item. See Section 6.1.

4.2.4. Use Case: Automated Daily Data Integrity Sweep

Property	Detail
Primary Actor	Temporal (time-based trigger at 00:00 local time)
Goal	Proactively manage assessment and task state without requiring user interaction.

Flow:

1. At 00:00 local time, an internal background service executes.
2. The service scans all Assessments and Tasks with a Pending or Upcoming status.
3. **Overdue Mutation:** If the current date has surpassed the item's dueDate, the system automatically transitions the status to Overdue.
4. The Analytics Engine triggers a targeted recalculation of the affected Module's calculated_late_count field.

5. If the user has notifications enabled and is not within configured Quiet Hours, the OS dispatches a local notification alerting the student to the overdue item.

4.3. Fully Attributed Entity Relationship Model

NOTE: All entities utilise string UUIDs as Primary Keys. Any property prefixed with `calculated_` is computed by the Analytics Engine (TileFormulaEvaluator) and persisted to the database — the UI only reads, never computes, these values.

Entity: Degree

The root domain entity representing the student's single academic programme.

- **id** (PK)
- **degreeName, institutionName, specialization, nqfLevel, duration** (Attributes)
- **totalCreditsToGraduate, status, startDate, expectedEndDate** (Attributes)
- **terms** (Array of AcademicTerm FKs)
- **Calculated (Analytics):** `calculated_total_registered_credits`, `calculated_credits_per_year`, `calculated_overall_gpa`, `calculated_degree_progress_percent`

Entity: AcademicTerm

A distinct academic period — year, semester, or quarter. Supports self-referencing hierarchy (e.g., a Year term containing Semester sub-terms).

- **id** (PK), **degreeId** (FK), **parentTermId** (Self-referencing FK)
- **academicYear, termName, startDate, endDate, gradePointAverageGoal, durationInWeeks, notionalHoursPerCredit**
- **Calculated (Analytics):** `calculated_term_gpa`, `calculated_term_navigator_data`, `calculated_cumulative_weight_timeline`, `calculated_total_weekly_study_goal`

Entity: Module (V7 Composite — Blueprint + Offering)

In V7, ModuleOffering and ModuleBlueprint are unified under a single Module domain object, simplifying state interactions while maintaining referential integrity in the underlying database.

- **offeringId** (PK), **moduleId** (Blueprint FK)
- **moduleCode, moduleName, credits, moduleType, anchorTermId, status**
- **minFinalGrade, targetFinalGrade, isContinuous, targetExamWeight**
- **gradeComposition** (Complex Object: Coursework vs Exam split, Pass Marks, Subminimum thresholds)
- **exam** (Decoupled ExamData Object — manages independent exam date, result, entrance status, and subminimum checks)
- **Calculated (Analytics):** `calculated_current_semester_grade`, `calculated_final_grade`, `calculated_gap_to_target`, `calculated_exam_entrance_gap`, `calculated_pass_fail_status`, `calculated_projected_final_grade`, `calculated_required_average_on_remaining`

Entity: Assessment

A single graded deliverable mapped under a Module. Assessments are the primary anchors for student performance data.

- **id** (PK), **moduleOfferingId** (FK)
- **assessmentName**, **assessmentType**, **weight (%)**, **dueDate**, **status**
- **result** (Value Object — GradePercentage, enforcing 0–100 integer validation)
- **effort** (Category Enum: Standard, High, Low)
- **strategyType** (Enum: SPRINT | MARATHON)
- **estimatedEffortMinutes**, **venue**
- **Calculated:** `calculated_time_spent`, `calculated_priority_score`

Entity: Task (NEW in V7)

Granular, actionable study checkpoints attached to a Module, optionally as children of a parent Assessment. Tasks represent the actual study sessions needed to achieve an Assessment result.

- **id** (PK), **moduleId** (FK), **assessmentId** (Optional FK), **parentTaskId** (Self-ref FK)
- **title**, **status**, **plannedTimeMinutes**, **dueDate**, **priority**, **manualPlacement**
- **strategyType** (Enum: SPRINT | MARATHON — mirrors parent Assessment strategy)
- **Calculated:** `calculated_time_spent`

KEY CONCEPT: Sprint Tasks are short, intense workloads (e.g., Complete Quiz 1). Marathon Tasks are long-term, multi-week efforts (e.g., Final Year Project). This distinction drives the Effort Spike detection algorithm in the Horizon Dashboard.

Entity: SystemSettings

A persisted singleton Domain Entity (not a UI preference store) that governs the entire application environment.

- **theme** ('light', 'dark', 'system')
- **language**, **filterMode**, **termDisplayMode**, **clockStyle**
- **lastSeenVersion** (Upgrade detection — triggers release notes or critical data migrations on version change)
- **lastBackupDate** (Tracks the most recent successful Google Drive sync)
- **notifications** (Complex NotificationPreferences object — master toggles, quiet hours, intensity levels, backup reminders, sound preferences)

5. Engineering Standards & Guardrails

The following standards are non-negotiable for all V7 development. They represent lessons codified from V1's architectural failures and are enforced via code review. Any Pull Request violating these standards must be rejected.

Standard	Rule	Rationale
Data Integrity	NEVER mutate data directly in the UI. Always dispatch a Command to the Application layer.	Database and Zustand store remain perfectly synchronised; no stale UI state.
UI Stability	React components must never be defined inside other components.	Prevents closure bugs and unnecessary re-mounts on every parent render.
Render Loops	Use <code>useEffectOnceWhen</code> and <code>useStableCallback</code> aggressively in complex data tables.	Eliminates infinite re-render loops that degrade mobile performance.
Manual Testing	Automated Jest tests are insufficient for native API coverage.	All features require Manual Testing Checklists run on target hardware before merging.
Schema Migrations	Migrations must be idempotent and forward-only (e.g., <code>005_add_hlc_columns.ts</code>).	Zero data loss on version upgrades; rollback path is not required by design.
State Writes	Commands flow: UI → CommandHandler → Repository → SQLite → Zustand update.	Transactional writes prevent partial saves and UI/DB desync.

5.1. Data Flow Mandate

All state changes in Academic-OS must flow through a single, deterministic pipeline:

DATA FLOW: UI Event → CommandHandler → Domain Model Validation → Repository → SQLite Write → Analytics Recalculation → Zustand Store Update → UI Re-render

This pipeline guarantees that the database and the UI store are always in sync. Bypassing any step — for example, updating Zustand directly without writing to SQLite — creates ghost state that will be overwritten on the next app boot.

5.2. Native API Testing Protocol

Because V7 interacts deeply with native APIs (Electron file system, Capacitor SQLite, Google OAuth intents), standard Jest unit tests are structurally insufficient to verify correct behaviour. The following protocol applies:

- **Unit Tests:** Domain Model logic, Value Object validation, Analytics Engine calculations.
- **Integration Tests:** Repository layer queries, migration scripts, CommandHandler flows.

- **Manual Hardware Checklists:** All features touching native APIs require explicit manual testing on a physical Android device and the Electron Desktop binary before merging to main.
 - **Boot Sequence Checklist:** Each release must pass a full cold-boot test on both platforms — verifying migration execution, theme application, and Zustand hydration in sequence.
-

6. Conclusion & Outstanding Issues

Academic-OS V7 represents a complete architectural evolution from a spreadsheet-replacement PWA into a robust, natively-executing personal academic operating system. By combining Clean Architecture, an offline-first SQLite strategy, a decoupled Analytics Engine, and native Electron and Capacitor runtimes, the application delivers a reliable, high-performance experience that actively empowers students to take control of their workload capacity and grade trajectories.

The transition from V1 eliminates the core pain points that limited it: volatile browser storage, in-component grade arithmetic, and lack of a mobile-native experience. V7's architecture is explicitly designed to accommodate future expansion without codebase degradation.

6.1. Outstanding Issues & Roadmap

The following items remain active architectural considerations or prioritised roadmap features:

1. **Cloud Sync Conflict Resolution:** Currently operating on a last-backup-wins strategy via Google Drive tokens. Future iterations require deeper Event Sourcing merge conflict resolution — specifically vector clock CRDT logic — for students who use desktop and mobile simultaneously while offline.
2. **University Calendar API Ingestion:** Expanding the CSV ingestion layer into a standardised iCal/ICS integration, enabling import of official university timetables directly into the Assessment timeline.
3. **Complex Prerequisite Forward Validation:** Explicitly notifying students that failing an active prerequisite module will cascade delays across the degree completion graph — forecasting delayed graduation dates.
4. **Capacitor Background Alarms (iOS/Android):** Implementing deep native OS background task notifications for due-in-1-hour tracking, given the aggressive suspension limits imposed on mobile apps when running offline in the background.
5. **Advanced Telemetry vs Privacy:** Resolving how to potentially crowdsource anonymised module difficulty data (e.g., average pass rates per course code) without breaching the strict offline-first, no-telemetry privacy philosophy.

— End of Document —